Community Detection in Complex Networks

Part 1: Introduction to Communities

1.1. Background

Many real-world networks—such as social networks, biological networks, and collaboration graphs—exhibit community structure, where nodes are densely connected within groups and sparsely connected between them. Detecting these communities helps in understanding the functional and structural properties of networks.

1.2. Key Concepts

- **Community:** A group of nodes with more internal than external connections.
- **Modularity:** A measure of the strength of division of a network into communities.
- **Overlapping vs. Non-overlapping Communities:** Whether a node can belong to multiple communities.
- **Hierarchical Clustering:** Building a tree of communities based on node similarity or connection strength.

Part 2: Hierarchical Clustering for Community Detection

2.1. Theoretical Background

Hierarchical clustering methods build a dendrogram of communities using either:

- **Agglomerative (bottom-up):** Start with each node as a community and merge similar ones.
- **Divisive (top-down):** Start with one community and recursively split.

Similarity can be measured using:

- **Jaccard similarity**
- **Cosine similarity**
- **Edge betweenness** (Girvan-Newman algorithm)

2.2. Implementation

Write a Python function `hierarchical_communities(graph, method='betweenness')` that:

1. Implements either agglomerative or divisive clustering.
2. For divisive method, uses edge betweenness to iteratively remove edges.
3. Returns a dendrogram and the best partition based on modularity.

Part 3: Modularity Optimization

3.1. Modularity Definition

Consider a network with N nodes and L links and a partition into n_c communities, each community having N_c nodes connected to each other by L_c links. Modularity is defined as:

$$M = \sum_{c=1}^{n_c} \left[\frac{L_c}{L} - \left(\frac{k_c}{2L} \right)^2 \right]$$

3.2. Louvain Method

Implement or use a library to apply the **Louvain method**:

- Greedily optimizes modularity by moving nodes and merging communities.
- Efficient for large networks.

Exercise:

- Apply the Louvain method to a real-world network (e.g., Zachary's Karate Club).
- Visualize the resulting communities.

Part 4: Overlapping Communities

4.1. Background

In many networks (e.g., social networks), nodes can belong to multiple communities. Methods like **CFinder** (Clique Percolation) and **BigClam** allow overlapping communities.

4.2. Clique Percolation Method (CPM)

Write a function `detect_overlapping_communities(graph, k=3)` that:

1. Finds all k-cliques in the network.
2. Builds a clique-clique overlap matrix.
3. Identifies connected components of cliques as communities.

Part 5: Characterizing Communities

5.1. Community Quality Metrics

- **Conductance:** Fraction of edges leaving the community.
- **Internal Density:** Ratio of internal edges to possible internal edges.
- **Average Degree Inside vs. Outside**

5.2. Role of Nodes in Communities

- **Hub:** High-degree nodes connecting communities.
- **Bridge:** Nodes connecting different communities.
- **Internal:** Nodes with most connections inside their community.

Exercise:

- Compute and visualize the roles of nodes in a detected community structure.